

Modified SAP-1 CPU in Verilog HDL

Computer Architecture Mini-Project

Abstract

This project implements a modified SAP-1 CPU in Verilog HDL. The processor uses an 8-bit datapath, 4-bit addresses, 16 memory locations, and an instruction format consisting of a 4-bit opcode and a 4-bit operand. The CPU supports LDA, ADD, SUB, MUL, DIV, OUT, and HLT instructions. Multiplication and division are implemented without using the Verilog `*` and `/` operators. Functional simulation verifies normal execution, arithmetic edge cases, division by zero, and halt behavior.

1 System Architecture and Datapath

The CPU follows a multi-cycle SAP-1 style architecture. Each instruction is processed through fetch, decode, and execute stages. A multiplexer-based 8-bit shared bus is used instead of an internal tri-state bus, which makes the design more suitable for FPGA synthesis.

Component	Verilog File
Program Counter	<code>program_counter.v</code>
MAR	<code>memory_address_register.v</code>
RAM	<code>ram_16x8.v</code>
Instruction Register	<code>instruction_register.v</code>
Accumulator	<code>accumulator.v</code>
B Register	<code>b_register.v</code>
ALU	<code>alu.v</code>
Output Register	<code>output_register.v</code>
Control Unit	<code>control_unit.v</code>
Bus Multiplexer	<code>bus_mux.v</code>
Top-level CPU	<code>sap1_top.v</code>
Testbench	<code>sap1_tb.v</code>

The top-level module connects the datapath, control unit, RAM programming interface, and external outputs:

```

1 module sap1_top(
2     input clk,
3     input reset,
4     input prog_we,
5     input [3:0] prog_addr,
6     input [7:0] prog_data,
7     output [7:0] out,
8     output halted,
9     output reg div_zero
10 );

```

Listing 1: Top-level interface from sap1_top.v

2 Module Descriptions

This section describes the function of each Verilog module and how it contributes to the SAP-1 datapath.

2.1 Program Counter

The Program Counter in `program_counter.v` is a 4-bit register that stores the address of the next instruction. During `FETCH_ADDR`, its value is selected onto the bus and loaded into the MAR. During `FETCH_INST`, the control unit asserts `pc_inc`, so the PC advances to the next instruction address. The module also includes reset and load support, although normal execution mainly uses reset and increment.

```

1 if (reset)
2     pc <= 4'h0;
3 else if (load)
4     pc <= data_in;
5 else if (inc)
6     pc <= pc + 4'h1;

```

Listing 2: Program Counter update logic

2.2 Memory Address Register

The Memory Address Register in `memory_address_register.v` stores the 4-bit address sent to RAM. During instruction fetch, it receives the PC value. During memory-based instructions such as LDA, ADD, SUB, MUL, and DIV, it receives the operand field from the instruction register. Therefore, the MAR is the address bridge between the control sequence and the RAM.

```

1 if (reset)
2     address <= 4'h0;
3 else if (load)
4     address <= data_in;

```

Listing 3: MAR load logic

2.3 RAM

The RAM in `ram_16x8.v` contains 16 memory locations, each 8 bits wide. It stores both program instructions and data values. The CPU reads RAM through the current MAR address. The testbench loads instructions and data through the programming interface `prog_we`, `prog_addr`, and `prog_data` before execution begins.

```

1 reg [7:0] memory [0:15];
2 assign data_out = memory[address];
3
4 always @(posedge clk) begin
5     if (prog_we)
6         memory[prog_addr] <= prog_data;
7 end

```

Listing 4: RAM storage and program loading

2.4 Instruction Register

The Instruction Register in `instruction_register.v` stores the 8-bit instruction fetched from RAM. It separates the instruction into two fields: `instruction[7:4]` as the opcode and `instruction[3:0]` as the operand. The opcode is sent to the control unit, while the operand is used as a RAM address for memory-reference instructions.

```

1 if (reset)
2     instruction <= 8'h00;
3 else if (load)
4     instruction <= data_in;
5
6 assign opcode = instruction[7:4];
7 assign operand = instruction[3:0];

```

Listing 5: Instruction Register field extraction

2.5 Accumulator

The Accumulator in `accumulator.v` is the main 8-bit working register of the CPU. LDA loads RAM data directly into the accumulator. Arithmetic instructions use the accumulator as the first ALU input, and the ALU result is written back into the accumulator. Thus, most data processing operations update this register.

```

1 if (reset)
2     data_out <= 8'h00;
3 else if (load)
4     data_out <= data_in;

```

Listing 6: Accumulator register logic

2.6 B Register

The B Register in `b_register.v` stores the second ALU operand. For ADD, SUB, MUL, and DIV, the CPU first loads `RAM[operand]` into the B register during `ALU_READ`. In the next state, the ALU operates on the accumulator and B register values.

```

1 if (reset)
2     data_out <= 8'h00;
3 else if (load)
4     data_out <= data_in;

```

Listing 7: B Register load logic

2.7 ALU

The ALU in `alu.v` performs ADD, SUB, MUL, and DIV operations based on the opcode. ADD and SUB use standard 8-bit arithmetic. MUL is implemented using shift-and-add logic, and DIV is implemented using restoring division based on shift/subtract operations. Division by zero is explicitly handled by returning `8'hFF` and asserting `div_zero`.

```

1 case (opcode)
2     OP_ADD: result = a + b;
3     OP_SUB: result = a - b;
4     OP_MUL: result = multiply_shift_add(a, b);
5     OP_DIV: begin
6         if (b == 8'h00) begin
7             result = 8'hFF;
8             div_zero = 1'b1;
9         end else begin
10            result = divide_restoring(a, b);
11        end
12    end
13    default: result = a;
14 endcase

```

Listing 8: ALU operation selection

2.8 Output Register

The Output Register in `output_register.v` stores the value produced by the OUT instruction. When OUT executes, the control unit selects the accumulator onto the bus and asserts `out_load`. The output register then holds this value as the external CPU output.

```

1 if (reset)
2     data_out <= 8'h00;
3 else if (load)
4     data_out <= data_in;

```

Listing 9: Output Register load logic

2.9 Control Unit

The Control Unit in `control_unit.v` is an FSM that coordinates all CPU operations. It generates the bus select signal, register load enables, PC increment signal, ALU opcode, and halted output. Its states implement instruction fetch, opcode decode, memory address setup, data read, ALU write-back, output write, and halt.

```

1 always @(posedge clk or posedge reset) begin
2     if (reset)
3         state <= FETCH_ADDR;
4     else
5         state <= next_state;
6 end
7
8 bus_sel = BUS_ZERO;
9 mar_load = 1'b0;
10 ir_load = 1'b0;
11 pc_inc = 1'b0;
12 a_load = 1'b0;
13 b_load = 1'b0;
14 out_load = 1'b0;

```

Listing 10: Control Unit state register and default controls

2.10 Bus Multiplexer

The Bus Multiplexer in `bus_mux.v` implements the shared 8-bit datapath. It selects one source at a time from PC, RAM data, IR operand, accumulator data, ALU result, or zero. The PC and operand are only 4 bits, so they are zero-extended to 8 bits before being placed on the bus.

```

1 case (sel)
2     BUS_PC:      bus_data = {4'h0, pc};
3     BUS_RAM:     bus_data = ram_data;
4     BUS_OPERAND: bus_data = {4'h0, ir_operand};
5     BUS_A:       bus_data = a_data;
6     BUS_ALU:     bus_data = alu_result;
7     default:     bus_data = 8'h00;
8 endcase

```

Listing 11: Bus source selection from `bus_mux.v`

3 Instruction Set and Opcode Format

Each instruction is 8 bits:

$$\text{Instruction} = \{\text{opcode}[3 : 0], \text{operand}[3 : 0]\}$$

Opcode	Instruction	Operation
1	LDA addr	$A = \text{RAM}[\text{addr}]$
2	ADD addr	$A = A + \text{RAM}[\text{addr}]$
3	SUB addr	$A = A - \text{RAM}[\text{addr}]$
4	MUL addr	$A = A * \text{RAM}[\text{addr}]$ using shift-add logic
5	DIV addr	$A = A / \text{RAM}[\text{addr}]$ using shift/subtract division
E	OUT	Output = A
F	HLT	Stop CPU execution
0	NOP	No operation/reserved

For example, `8'h4B` is interpreted as `MUL RAM[0xB]`.

4 ALU Design

The ALU in `alu.v` is a combinational module. It receives the accumulator value `a`, the B register value `b`, and the current opcode. It outputs an 8-bit `result` and a `div_zero` flag. The ALU supports four arithmetic instructions: ADD, SUB, MUL, and DIV.

```
1 module alu(  
2     input [7:0] a,  
3     input [7:0] b,  
4     input [3:0] opcode,  
5     output reg [7:0] result,  
6     output reg div_zero  
7 );
```

Listing 12: ALU module interface

4.1 ADD and SUB

ADD and SUB use standard 8-bit binary arithmetic. ADD computes $A + B$, while SUB computes $A - B$. Because the datapath is 8 bits wide, overflow and underflow wrap around naturally. For example, $250 + 10$ becomes 4, and $3 - 10$ becomes 249.

```
1 case (opcode)  
2     OP_ADD:  
3         result = a + b;  
4     OP_SUB:  
5         result = a - b;  
6 endcase
```

Listing 13: ADD and SUB selection

4.2 Multiplication Without *

MUL uses the shift-and-add multiplication algorithm. The multiplier is checked bit by bit. If a multiplier bit is 1, the current shifted multiplicand is added to the product. The multiplicand is shifted left every iteration. This avoids the Verilog `*` operator while still implementing multiplication in hardware. The internal product is 16 bits, but only the lower 8 bits are returned to match the CPU datapath.

The algorithm begins by clearing the product register. It then scans the multiplier from bit 0 to bit 7. For each bit, the shifted multiplicand is added only when that multiplier bit is high. After each iteration, the multiplicand is shifted left by one position. This is equivalent to binary long multiplication and is suitable for this project because the 8-bit operand width gives a fixed loop count of eight iterations.

Insert shift-and-add multiplication algorithm figure here.

Figure 1: Shift-and-add multiplication algorithm used for MUL

```
1 product = 16'h0000;
2 shifted_multiplicand = {8'h00, multiplicand};
3
4 for (i = 0; i < 8; i = i + 1) begin
5     if (multiplier[i])
6         product = product + shifted_multiplicand;
7     shifted_multiplicand = shifted_multiplicand << 1;
8 end
9
10 multiply_shift_add = product[7:0];
```

Listing 14: Shift-and-add multiplication from alu.v

4.3 Division Without /

DIV uses the restoring division algorithm. The quotient is built one bit at a time from the most significant bit to the least significant bit. At each step, the partial remainder is shifted left and compared with the divisor. If the partial remainder is large enough, the divisor is subtracted and the quotient bit is set to 1. This avoids the Verilog / operator and produces integer division, so any remainder is discarded.

The algorithm first initializes the quotient and remainder to zero. It then processes the dividend from bit 7 down to bit 0. At every iteration, the next dividend bit is shifted into the partial remainder. If the partial remainder is greater than or equal to the divisor, the divisor is subtracted and the corresponding quotient bit is set. Otherwise, the quotient bit remains zero. This produces an integer quotient and discards the final remainder.

Insert restoring division algorithm figure here.

Figure 2: Restoring division algorithm used for DIV

```
1 for (i = 7; i >= 0; i = i - 1) begin
2     remainder = {remainder[7:0], dividend[i]};
3     if (remainder >= {1'b0, divisor}) begin
4         remainder = remainder - {1'b0, divisor};
5         quotient[i] = 1'b1;
6     end
7 end
```

Listing 15: Restoring division from alu.v

4.4 Division by Zero

If the divisor is zero, the ALU returns 8'hFF and asserts `div_zero`:

```
1 if (b == 8'h00) begin
2     result = 8'hFF;
3     div_zero = 1'b1;
4 end else begin
5     result = divide_restoring(a, b);
6 end
```

Listing 16: Division-by-zero handling

4.5 Opcode-Based Result Selection

The final ALU output is selected using the opcode. For unsupported opcodes, the ALU passes the accumulator value through unchanged.

```

1  case (opcode)
2      OP_ADD: result = a + b;
3      OP_SUB: result = a - b;
4      OP_MUL: result = multiply_shift_add(a, b);
5      OP_DIV: begin
6          if (b == 8'h00) begin
7              result = 8'hFF;
8              div_zero = 1'b1;
9          end else begin
10             result = divide_restoring(a, b);
11         end
12     end
13     default: result = a;
14 endcase

```

Listing 17: Complete ALU operation selection

5 Control Unit FSM

The control unit is a finite state machine. It drives the datapath by selecting the bus source and enabling the correct register at each clock cycle.

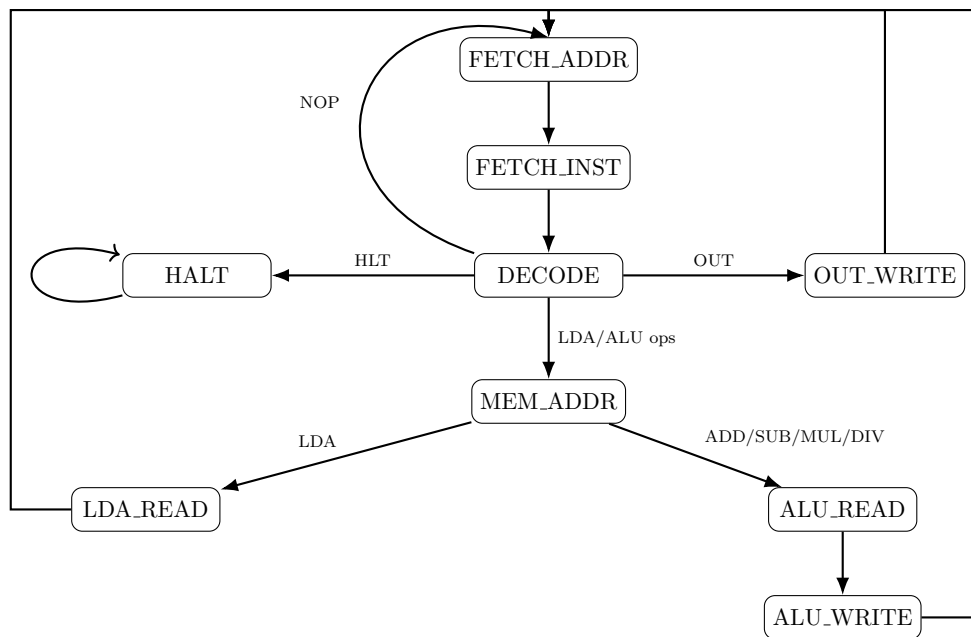


Figure 3: FSM state diagram of the SAP-1 control unit

State	Purpose
FETCH_ADDR	Put PC on bus and load MAR.
FETCH_INST	Read RAM[MAR] into IR and increment PC.
DECODE	Select the execution path from opcode.
MEM_ADDR	Load operand address into MAR.
LDA_READ	Load RAM data into Accumulator.
ALU_READ	Load RAM data into B Register.
ALU_WRITE	Write ALU result back to Accumulator.
OUT_WRITE	Copy Accumulator to Output Register.
HALT	Assert halted and stop sequencing.

```

1  FETCH_ADDR: begin
2      bus_sel = BUS_PC;
3      mar_load = 1'b1;
4      next_state = FETCH_INST;
5  end
6
7  FETCH_INST: begin
8      bus_sel = BUS_RAM;
9      ir_load = 1'b1;
10     pc_inc = 1'b1;
11     next_state = DECODE;
12 end
13
14 DECODE: begin
15     case (opcode)
16         OP_HLT: next_state = HALT;
17         OP_OUT: next_state = OUT_WRITE;
18         OP_NOP: next_state = FETCH_ADDR;
19         default: next_state = MEM_ADDR;
20     endcase
21 end

```

Listing 18: Fetch and decode examples from control_unit.v

6 Simulation Results

The testbench loads this program into RAM:

```

1 write_memory(4'h0, 8'h18); // LDA RAM[0x8]
2 write_memory(4'h1, 8'h29); // ADD RAM[0x9]
3 write_memory(4'h2, 8'h3A); // SUB RAM[0xA]
4 write_memory(4'h3, 8'h4B); // MUL RAM[0xB]
5 write_memory(4'h4, 8'h5C); // DIV RAM[0xC]
6 write_memory(4'h5, 8'hE0); // OUT
7 write_memory(4'h6, 8'hF0); // HLT

```

Listing 19: Main program loaded in sap1_tb.v

The data values are $\text{RAM}[8] = 10$, $\text{RAM}[9] = 3$, $\text{RAM}[A] = 4$, $\text{RAM}[B] = 2$, and $\text{RAM}[C] = 3$. Therefore:

$$((10 + 3 - 4) \times 2) / 3 = 6$$

Simulation confirms that the output register becomes `8'd6`, the CPU halts, and `div_zero` remains low for the normal program. Additional tests verify CPU division by zero, multiplication overflow, addition overflow, subtraction underflow, integer division truncation, LDA overwrite behavior, and HLT stop behavior. All test cases pass in XSim.

7 Design Decisions, Challenges, and Conclusion

The main design decision was to use a bus multiplexer instead of internal tri-state buffers. This keeps the design FPGA-friendly and avoids multiple drivers on the internal bus. The CPU was also separated into small modules to make the datapath and control logic easier to test and explain.

The main challenge was implementing MUL and DIV without `*` and `/`. Shift-and-add multiplication and restoring division were used because they are clear, bounded to 8 iterations, and suitable for a simple hardware implementation. The FSM was also important because each instruction requires several clock cycles rather than executing in one cycle.

In conclusion, the modified SAP-1 CPU demonstrates the main concepts of a simple multi-cycle processor: instruction fetching, decoding, operand access, ALU execution, output transfer, and halt control. The design was implemented using separate Verilog modules for the PC, MAR, RAM, IR, registers, ALU, output register, bus multiplexer, and FSM-based control unit. The completed CPU supports LDA, ADD, SUB, MUL, DIV, OUT, and HLT instructions using an 8-bit datapath and 4-bit address space. Simulation results show that the processor executes the main program correctly and handles important edge cases such as arithmetic overflow, underflow, integer division truncation, division by zero, and halt behavior. Therefore, the project satisfies the required SAP-1 architecture goals while also extending the basic SAP-1 model with multiplication and division operations.